

**SCOPE,
ENVIRONMENT,
CONTEXT**

QUESTION

```
const age = 22;  
const isAllowed = age >= 18 ? 'Yes' : 'No';  
  
console.log(isAllowed ); // ?
```

ANSWER

```
const age = 22;  
const isAllowed = age >= 18 ? 'Yes' : 'No';  
  
console.log(isAllowed ); // 'Yes'
```

QUESTION

```
let age;  
  
console.log(age); // ?  
  
age = +prompt('Enter age');  
  
console.log(age); // ?
```

ANSWER

```
let age;  
  
console.log(age); // undefined  
  
age = +prompt('Enter age'); // '54' -> +'54' -> 54  
  
console.log(age); // 54
```

QUESTION

```
console.log(age); // ?  
  
const age = +prompt('Enter age');  
  
console.log(age); // ?
```

ANSWER

```
console.log(age); // Reference error: age is not defined  
  
const age = +prompt('Enter age');  
  
console.log(age); // ?
```

QUESTION

```
console.log(age); // ?  
  
var age = +prompt('Enter age');  
  
console.log(age); // ?
```

ANSWER

```
console.log(age); // undefined  
  
var age = +prompt('Enter age'); // '12' -> +'12' -> 12  
  
console.log(age); // 12
```

QUESTION

```
const age = +prompt('Enter age');

if (age >= 14 && age <= 18) {
  const message = 'You are too little';
} else if (age > 18) {
  const message = 'You are an adult';
} else {
  console.log('Enter age again');
}

console.log(message); // ?
```

ANSWER

```
const age = +prompt('Enter age');

if (age >= 14 && age <= 18) {
  const message = 'You are too little';
} else if (age > 18) {
  const message = 'You are an adult';
} else {
  console.log('Enter age again');
}

console.log(message); // Reference Error: message is not defin
```

QUESTION

```
var age = +prompt('Enter age');

if (age >= 14 && age <= 18) {
  var message = 'You are too little';
} else if (age > 18) {
  var message = 'You are an adult';
} else {
  console.log('Enter age again');
}

console.log(message); // ?
```

ANSWER

```
var age = +prompt('Enter age'); // '16' -> +'16' -> 16

if (age >= 14 && age <= 18) {
  var message = 'You are too little';
} else if (age > 18) {
  var message = 'You are an adult';
} else {
  console.log('Enter age again');
}

console.log(message); // You are too little
```

QUESTION

```
const name = 'Ivan';

function sayHi() {
  console.log(`Hi ${name}`); // ?
}

sayHi();
```

ANSWER

```
const name = 'Ivan';

function sayHi() {
  console.log(`Hi ${name}`); // 'Hi Ivan'
}

sayHi();
```

QUESTION

```
function sayHi() {  
  console.log(`Hi ${name}`); // ?  
}  
  
const name = 'Ivan';  
  
sayHi();
```

ANSWER

```
function sayHi() {  
  console.log(`Hi ${name}`); // 'Hi Ivan'  
}  
  
const name = 'Ivan';  
  
sayHi();
```

QUESTION

```
function sayHi() {  
  const message = `Hi ${name}`;  
}  
  
const name = 'Ivan';  
  
sayHi();  
  
console.log(message); // ?
```

ANSWER

```
function sayHi() {  
  const message = `Hi ${name}`;  
}  
  
const name = 'Ivan';  
  
sayHi();  
  
console.log(message); // Reference Error: message is not defined
```

QUESTION

```
function sayHi() {  
  message = `Hi ${name}`;  
}  
  
const name = 'Ivan';  
let message;  
  
sayHi();  
  
console.log(message); // ?
```

ANSWER

```
function sayHi() {  
  message = `Hi ${name}`;  
}  
  
const name = 'Ivan';  
let message;  
  
sayHi();  
  
console.log(message); // 'Hi Ivan'
```

SCOPE, ENVIRONMENT

DEFINITION

***Scope** is the current context of execution in which values and expressions are "visible" or can be referenced.*

<https://developer.mozilla.org/en-US/docs/Glossary/Scope>

SCOPE TYPES

- **Global scope:** The default scope for all code running in script mode
- **Module scope:** The scope for code running in module mode.
- **Function scope:** The scope created with a function.

In addition, identifiers declared with certain syntaxes, including `let`, `const`, `class`, or (in strict mode) `function`, can belong to an additional scope:

- **Block scope:** The scope created with a pair of curly braces (a block).

GLOBAL SCOPE

```
var x = 1;

function print() {
  console.log('print');
}

console.log('global', x); // a variable is accesed in global s
print(); // a function is accesed in global scope by an identi
```

MODULE SCOPE

IIFE (Immediately Invoked Function Expression) is a JavaScript function that runs as soon as it is defined

<https://developer.mozilla.org/en-US/docs/Glossary/IIFE>

```
(function () {  
    // inside IIFE  
})();
```

MODULE SCOPE

```
var x = 1;

(function () {
  // Module Scope
  var x = 2; // inside module scope identifier 'x' is not the
  var y = 5; // identifier with name 'y' is accessbile only in

  console.log('iffe', x, y); // 'iffe 2 5'
})();

// identifiers with names 'x' and 'y' are accessible in global
console.log('global', x, y); // Reference error: 'y' is not de
```

FUNCTION SCOPE

```
var x = 1;

function print() {
  // Function scope
  var y = 2;
  // identifiers with names 'x' and 'y' are accessible in func
  console.log('print', x, y); // 'print 1 2'
  // A question - how is 'x' accessible in function scope from
}

print(); // a function is accesed in global scope by an identi
console.log(y); // Reference error: 'y' is not defined
```

BLOCK SCOPE

```
const y = 2;

if (true) {
  // Block scope
  var x = 1; // Identifier defined with 'var' is not being aff
  let z = 3;
  console.log(x, y, z); // 1 2 3
}

{
  // Block scope
  let z = 5;
  console.log(x, y, z); // 1 2 5
}
```

EXECUTION CONTEXT

DEFINITION

***Execution context** defines environment
in which our code is executed*

It contains a lot of internal components that the engine uses to keep track of the execution flow of the code

DEFINITION

***Environment Record** is being used to keep track and maintain the identifier bindings that have been created for the variable and function declarations*

JAVASCRIPT COMPILER PHASES

When a global code / function / eval is executed it goes through the below phase:

1. Memory Creation phase
2. Code Execution phase

MEMORY CREATION PHASE

During this stage, JavaScript prepares the groundwork by allocating memory for variables and functions, scanning for declarations, and setting up the stage for the code execution. In the memory creation phase, the execution context stores the values of variables and functions in key-value pairs.

MEMORY CREATION PHASE

Source code for memory creating phase

```
var number = 1;  
  
function printGlobal() {  
    console.log(number);  
}  
  
printGlobal();
```

After memory creating phase

```
number: undefined;  
printGlobal: function;
```

CODE EXECUTION PHASE

Code Executed line by line. If there is a function call, then a new execution context is being created (memory creation phase) and then code inside function is being executed (code execution phase).

```
var number = 1;

function printGlobal() {
  console.log(number);
}

printGlobal();
```

After function code execution is completed, its execution context is deleted and the JavaScript continues to run code after this function call.

DEFINITION

Hoisting - is a jargon term for the part of memory creating phase when variables declared in the execution context by the keywords `var`, `const`, `let`, classes, function are hoisted. So the memory is allocated for them but they remain uninitialized (except `var`, for them their value is undefined).

They are initialized later during the code execution phase when JavaScript reaches their line of declaration.

If you try to access a variable before it's declared, this will result in the `ReferenceError`.

DEFINITION

Scope chain - is a mechanism made available through the `[[OuterEnv]]` property a possibility to find a variable in the parent context environment if it's not available in the current execution context

DEFINITION

Closures - *are formed whenever an inner function keeps a reference to the outer functions variables*

DEMO

USED MATERIALS

<https://www.youtube.com/watch?v=eghLLFbTDYM>

<https://dev.to/jahid6597/javascript-execution-context-a-deep-dive-4kno>

<https://developer.mozilla.org/en-US/docs/Glossary/Scope>

<https://developer.mozilla.org/en-US/docs/Glossary/IIFE>

<https://www.youtube.com/watch?v=zdGfo6l1yrA>

[https://en.wikipedia.org/wiki/Scope_\(computer_science\)](https://en.wikipedia.org/wiki/Scope_(computer_science))